

QuickCheck – en sorteringsfunksjon

- ▶ Installer pakken ved å kjøre – i terminal – komando
 - > `cabal install --lib QuickCheck`
 - muligens, last ned modulen QuickCheck.hs
 - https://wiki.haskell.org/Introduction_to_QuickCheck1
 - <http://www.cse.chalmers.se/~rjmh/QuickCheck/manual.html>
 - ▶ `import Test.QuickCheck` – ved starten av programfilen
- ```
qs (x:z) = qs[y | y<- z, y<x] ++ [x] ++ qs[y | y<- z, y>=x]
```
- Er resultatet sortert?
- ▶ `sorted [] = True`  
`sorted [x] = True`  
`sorted (x:y:xs) = x <= y && sorted (y:xs)`
  - > `quickCheck (\xs -> sorted (qs xs))`  
`+++ OK, passed 100 tests.`
  - ▶ quickCheck genererer tilfeldige testdata for de fleste typene (spesielt, for basistyper som Int, [Int], Char, String)

## Typer - Kap.3 (og senere 8.1-8.5, [8.6-8.7])

- ▶ En type representerer en mengde verdier

Eksempler:

- ▶ **heltall**  $\dots, -2, -1, 0, 1, 2, \dots$ , type `Int`
  - ▶ **par** av heltall, type `(Int, Int)`
  - ▶ **lister** av par av heltall, type `[(Int, Int)]`
  - ▶ **funksjoner** fra heltall til tegn, type `Int -> Char`
  - ▶ **egendefinerte typer** som `data Mybool = Usann | Sann`
- 
- ▶ Hvert Haskell uttrykk har en type: `<uttrykk> :: <type>`  
`f :: [Int] -> Int` er et lovlig 'uttrykk' i programfilen  
for en `f` som ellers er definert.  
  
`> :t uttrykk` (evt. `:type uttrykk`) angir typen i GHCi
- 
- ▶ Typen endres ikke under evaluering

# Typefeil

Anvendelse av en funksjon på et argument med feil type, f.eks.:

> 1 + False

No instance for (Num Bool) arising from a use of '+'

(+) forventer to tall, mens False er en logisk verdi

> 1 + false

Not in scope: 'false'

▶ Alle typefeil er funnet ved kompilering

som gjør programmer mer pålitelige og raskere (siden typesjekking under kjøring blir unødvendig).

## Noen kjøretidssfeilmeldinger (run-time errors) **Kompileringsfeilmeldinger stammer fra feil typing**

- ▶ gi siste elementet i listen last :: [a] -> a

last [x] = x

last (x:y:xs) = last (y:xs)

- > last [] = ??? **\*\*\*Exception: Prelude.last : empty list**

evt. lst [] = [] lst :: [a] -> [a]

lst [x] = [x]

lst (x:y:xs) = lst (y:xs)

- ▶ lst xs = xs!!(length xs - 1)  
**Exception. Prelude.!!: index too large ...**

- ▶ del liste i 2 like (+/- 1 element) deler

del [] = [] ([], []) del :: [a] -> ([a], [a])

del xs = **let** h = (length xs) 'div' 2 **in** (take h xs, drop h xs)

x 'div' y == div x y ==  $\lfloor \frac{x}{y} \rfloor$

**error: Couldn't match expected type ...**

# Typer

- ▶ Type-sjekking og type-avledning behandles senere (litt avhengig av tiden)
  - her legger vi bare frem de viktigste type-fenomenene
- ▶ Haskell klarer ofte å avlede en type til et uttrykk, med mindre
  1. uttrykket er galt (oftest), eller
  2. uttrykket er OK men meget komplisert (sjeldent)
- ▶ I det sjeldne tilfelle 2 skal man legge til noen type-informasjon

## Basistyper, hver med sett av operatører

- ▶ heltall `..., -2, -1, 0, 1, 2, ..., .....` type `Int` og `Integer`  
`+, -, *, div, mod`
- ▶ boolske verdier `True, False, .....` type `Bool`  
`not, ||, &&, or, and`
- ▶ tegn `..., 'A', ..., 'z', ..., .....` type `Char`  
`toUpper, toLower` i `Data.Char`
- ▶ strenger som `'abc'`, `.....` type `String`  
`++, head, tail,`
- ▶ brøktall som `1.2, 1.234.....` type `Float` og `Double`  
`+, -, *, /`

## Flere operatører

Basistyper kommer også med operatører som returnerer verdier av en annen type

- ▶ Float:

`==,<,...:: (Float,Float) -> Bool`

- ▶ Int:

`==,<,...:: (Int, Int) -> Bool`

- ▶ String:

`length :: [a] -> Int,`

`==,<,...:: (String,String) -> Bool`

- ▶ Char:

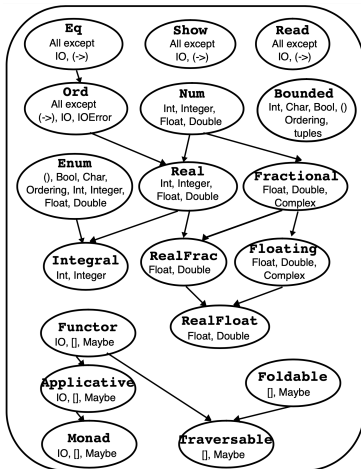
`ord:: Char->Int, chr:: Int -> Char,`

`isDigit:: Char -> Bool`

# Eksempler – verdi :: type

1 :: Int Num t => t

- Int, Float er typer
- Num, Fractional, Eq er typeklasser.



## Typer er (oftest) instanser av typeklasser (kap.8.5)

- ▶ **Show**, med en operasjon `show :: a -> String` (viser ikke!)
- Read**, med en operasjon `read :: String -> a` (leser ikke!)
- ▶ **Eq**, med `==, /= :: a -> a -> Bool`
- ▶ **Num**, `(+)`, `(-)`, `(*)`, `abs`, `negate...`,
  - en subklasse av **Eq** og **Show** med subklasser som
    - ▶ **Integral**, **Floating**  $\subset$  **Fractional**, ...
    - > `:t 1`  
`Num a => a`
    - og instanser: `Int`, `Integer` (**Integral**), `Float`, `Double` (**Floating**), `Rational` (**Fractional**) i `Data.Ratio`
- ▶ **Ord**, en subklasse av **Eq**, med totalt ordnet instanser (typer)  
`(<)`, `(<=)`, `(>)`, `(>=)` `:: a -> a -> Bool`  
`min,max :: a -> a -> a`
  - > `:t (<)`  
`Ord a => a -> a -> Bool`
  - ▶ ...
- ▶ Alle basistyper er instanser av **Eq**, **Ord**, **Show** og **Read**. Også lister og tupler er det, hvis så er typen av deres elementer.

## Eksempler – verdi :: type

|                                |    |                                             |                        |
|--------------------------------|----|---------------------------------------------|------------------------|
| 1                              | :: | Int                                         | Num t => t             |
| 2.0 + 1.2                      | :: | Float                                       | Fractional t => t      |
| [1, 1.2]                       | :: | [ Float ]                                   | Fractional t => [t]    |
| head [1, 1.2]                  | =  | 1.0 ::                                      | Fractional t => t      |
| -(-1)                          | :: | Int                                         | Num t => t             |
| - - 1                          | :: | feil                                        |                        |
| [1,2,3]                        | :: | [Int]                                       | Num t => [ t ]         |
| 'a'                            | :: | Char                                        |                        |
| 'a' : 'b' : []                 | :: | [Char]                                      |                        |
| "abc"                          | :: | [Char]                                      | dvs. String            |
| [ "ab", "ab", "cd" ]           | :: | [ [Char] ]                                  | dvs. [String]          |
| ( "ab", 12 )                   | :: | ( String, Int )                             | Num t => ( String, t ) |
| [ "ab", 12 ]                   | :: | feil – listeelementer må ha én type : [ t ] |                        |
| [ ( "ab", 12 ), ( "cd", 24 ) ] | :: | [(String,Num)]                              |                        |
| 1 + x                          | :: | feil                                        |                        |
| (1 +)                          | :: | Int -> Int                                  | Num a => a -> a        |

# Typer og Type-operatorer

- ▶ Lister: `[typeE]`
- ▶ Produkt (kartesisk): `(typeV, typeH)`
- ▶ Funksjoner: `typeK -> typeM`
- ▶ Syntaks for typer (typeuttrykk):  
$$T ::= \text{navn} \mid [T] \mid (\{T, \} T, T) \mid T \rightarrow T \mid (T \rightarrow T)$$
$$\text{navn} ::= \text{Int} \mid \text{Bool} \mid \dots \mid \text{Var} \mid \text{Brukerdefinert}$$
- ▶ **Tvetydig**: hvilken pil er hovedoperatoren i `a -> (b -> c)` ?  
`a -> b -> c -> d` forkorter `a -> (b -> (c -> d))`

# Lister

- ▶ Gitt `typeE` har vi en type av lister `[typeE]`
- ▶ Konstruksjon: `[]` og  
for `x :: typeE`, `l :: [typeE]`, blir `x:l :: [typeE]`  
`["a", "bc"] :: [String]`  
`[1, 1.2] == 1 : 1.2 : [] :: (Fractional t) => [t]`
- ▶ Destruksjon: `head (x:l) = x`      `tail (x:l) = l`
- ▶ Lengdefunksjon: `length : [a] -> Int`      polymorf!
- ▶ Prøv: `let e=1 in [e]==[1]`, `let e=12 in [e]==[1,2]`
- ▶ Alternativ notasjon  
`[1,2,3] = 1:[2,3] = 1:2:[3] = 1:2:3:[]`
- ▶ **Likhet**: hvis `typeE` har `==`, så har `[typeE] ==`,  
nemlig: lik lengde og elementvis likhet
- ▶ Hvis `typeE` er en instans av `Show`, så er `[typeE]`.
- ▶ Hvis `typeE` er en instans av **Ord**, så er `[typeE]`.

## Kartesisk produkt (par/tupler)

- ▶ Gitt `typeV` og `typeH` har vi en type av par `(typeV, typeH)`
- ▶ Konstruksjon: for `x :: typeV`, `y :: typeH`,  
blir `(x,y) :: (typeV, typeH)`  
`('a' , True) :: (Char , Bool )`  
`(1 , 1.2) :: (Num t, Fractional t1) => (t, t1)`
- ▶ Destruksjon: `fst (x,y) = x`    `snd (x,y) = y`  
polymorfe!
- ▶ **Likhet**: hvis `typeV` og `typeH` har `==`, så har `(typeV, typeH)`  
`==`, nemlig: `(x,y) == (x',y')` hviss `x==x'` og `y==y'`
- ▶ Hvis `typeV` og `typeH` er begge instanser av `Show`, så er  
`(typeV, typeH)`.
- ▶ Hvis `typeV` og `typeH` er begge instanser av **Ord**, så er  
`(typeV, typeH)`.

## Noen eksempler...

1. `['a','b','c']` :: ? `[Char]` / `String`
2. `"abc"` :: ? `[Char]` / `String`
3. `('a','b','c')` :: ? `(Char, Char, Char)`
4. `[ (False, '0'), (True, '1') ]` :: ? `[ (Bool, Char) ]`
5. `[ (False, 0), (True, 1) ]` :: ? `[ (Bool, Int) ]`
6. `( [False, 0], [True, 1] )` :: ? typefeil!
7. `( [False, True], ['0','1'] )` :: ? `( [Bool], [Char] )`

# Funksjonstyper

- ▶ Gitt  $\text{typeK}$  og  $\text{typeM}$  har vi en type  $\text{typeK} \rightarrow \text{typeM}$
- ▶ Konstruksjon: for  $x :: \text{typeK}$ ,  $\langle \text{uttrykk} \rangle :: \text{typeM}$   
 $\langle \text{navn} \rangle \ x = \langle \text{uttrykk} \rangle$   
– f.eks.:  $\text{plus } x \ y = x+y$  eller  
 $\backslash \ x \rightarrow \langle \text{uttrykk} \rangle$  (anonym funksjon)  
– f.eks.:  $\backslash \ x \rightarrow x+x \quad :: \text{Num } t \Rightarrow t \rightarrow t$   
– f.eks.:  $\backslash \ x \rightarrow \backslash \ y \rightarrow x+y \quad :: \text{Num } t \Rightarrow t \rightarrow t \rightarrow t$
- ▶  $f \ g = \backslash \ x \rightarrow (g \ x) + 1$   
 $f = \backslash \ g \ x \rightarrow (g \ x) + 1 \quad \backslash \ a \ b \rightarrow \dots == \backslash \ a \rightarrow (\backslash \ b \rightarrow \dots)$   
 $f \ g \ x = (g \ x) + 1$   
 $f :: \text{Num } a \Rightarrow (t \rightarrow a) \rightarrow t \rightarrow a$
- ▶ Destruksjon: funksjonsanvendelse,  $\langle \text{funksjon} \rangle \ \langle \text{uttrykk} \rangle$ :  
–  $(\text{plus } 3) \ 1 \quad (\backslash \ x \rightarrow x+x) \ 2 == 4$   
–  $f \ (\backslash \ y \rightarrow y*y) \quad \backslash \ x \rightarrow ((\backslash \ y \rightarrow y*y) \ x) + 1$   
 $== \backslash \ x \rightarrow x*x + 1$
- ▶ **Likhet** prinsipielt umulig
- ▶ **Show** prinsipielt umulig

## Basistyper; hver med sett av operatoreroperatorer er også typeelementer

- ▶ heltall `..., -2, -1, 0, 1, 2, ..., .....` :: `Int`  
`+, -, *, div, mod (+), (-), (*), (div) :: Num a => a -> a -> a`
- ▶ boolske verdier `True, False, .....` :: `Bool`  
`not, ||, &&, or, and not :: Bool -> Bool; or, and :: [Bool] -> Bool;`  
`(||), (&&) :: Bool -> Bool -> Bool`
- ▶ tegn `..., 'A', ..., 'z', ..., .....` :: `Char`  
`toUpper, toLower, isDigit (Bool) i Data.Char toUpper :: Char -> Char, isDigit :: Char -> Bool`
- ▶ strenger som `'abc', .....` :: `String`  
`++, head, tail, head :: [a] -> a, (++) :: [a] -> [a] -> [a]`
- ▶ brøktall `... 1.2, 3.5... .....` :: `Float`  
`+, -, *, / (/) :: Fractional a => a -> a -> a`

# Typer

- ▶ innebygde basistyper: Bool, Char, Int, ...
- ▶ listetype:  $[t]$ , for enhver type  $t$   
konstruert med:  $[]$  og  $:$
- ▶ partype:  $(t,p)$ , for alle typer  $t$  og  $p$   
konstruert med:  $(-, -)$
- ▶ funksjonstype:  $t \rightarrow p$  eller  $(t \rightarrow p)$ , for alle typer  $t$  og  $p$   
konstruert med definsjoner  $f\ x = \dots$  eller  $\lambda\ x \rightarrow \dots$
- ▶ Syntaks for typeuttrykk ("navn"):  
$$T ::= \text{navn} \mid [T] \mid (\{T, \} T, T) \mid T \rightarrow T \mid (T \rightarrow T)$$
$$\text{navn} ::= \text{Int} \mid \text{Bool} \mid \dots \mid \text{Var} \mid \text{Brukerdefinert}$$

## Noen eksempler...

- $$\backslash x y \rightarrow x y \quad :: \quad ? (a \rightarrow b) \rightarrow a \rightarrow b$$
- $$\backslash x y z \rightarrow (x y) z \quad :: \quad ? (a \rightarrow b \rightarrow c) \rightarrow a \rightarrow c$$

$$\backslash x y z \rightarrow x y z \quad :: \quad ? (a \rightarrow b \rightarrow c) \rightarrow a \rightarrow c$$
- $$\backslash x y z \rightarrow x (y z) \quad :: \quad ? (a \rightarrow b) \rightarrow (c \rightarrow a) \rightarrow b$$

$$(\backslash x \rightarrow \text{utt}(x)) \backslash y \rightarrow \text{utt}(x, y) \text{ a } b = \text{utt}(a)(\backslash y \rightarrow \text{utt}(a, y))$$

$$(((\backslash x y z \rightarrow x (y z)) (+1)) (+2)) 5 =$$

$$(\backslash y z \rightarrow (+1) (y z)) (+2) 5 =$$

$$(\backslash z \rightarrow (+1) ((+2) z)) 5 = (+1) ((+2) 5)$$

$$= 1 + ((+2) 5)$$

$$= 1 + (2 + 5) = 8$$

$$(((\backslash x y z \rightarrow x (y z)) (+1)) (*2)) 5 = 11$$
- $$[\text{tail}, \text{init}, \text{reverse}] \quad :: \quad ? [ [a] \rightarrow [a] ]$$

$$\text{tail} \quad :: \quad ? [a] \rightarrow [a]$$

$$\text{init} \quad :: \quad [a] \rightarrow [a]$$

$$\text{reverse} \quad :: \quad [a] \rightarrow [a]$$

# Curring (Haskell Curry)



uncurry  $\gg$   
 $\ll$  curry

| prefiks                                                                      |       | infiks                        |
|------------------------------------------------------------------------------|-------|-------------------------------|
| $\text{div} :: \text{Integral } t \Rightarrow t \rightarrow t \rightarrow t$ | $\gg$ | $\text{'div'}$ skrives infiks |
| $\text{div } 7 \ 2 == 3$                                                     |       | $7 \ \text{'div'} \ 2 == 3$   |

|                                                                   |       |                                                        |
|-------------------------------------------------------------------|-------|--------------------------------------------------------|
| $(+) :: \text{Int} \rightarrow \text{Int} \rightarrow \text{Int}$ | $\ll$ | $+ :: (\text{Int}, \text{Int}) \rightarrow \text{Int}$ |
| $(+) \ 3 \ 4 == 7$                                                |       | $3 + 4 == 7$                                           |
| $(-) :: \text{Int} \rightarrow \text{Int} \rightarrow \text{Int}$ | $\ll$ | $- :: (\text{Int}, \text{Int}) \rightarrow \text{Int}$ |

To typer er isomorfe:

$A \rightarrow (B \rightarrow C) \simeq (A, B) \rightarrow C$

$(f) :: A \rightarrow (B \rightarrow C) \simeq f :: (A, B) \rightarrow C$

$(f) \ a \ b \sim a \ \text{'f'} \ b \sim f(a,b)$

$(+2) = (+) \ 2 :: \text{Int} \rightarrow \text{Int}$

“section” (seksjon/snitt)

$(+2) \ 1 = 3$ , etc.

# Brukerdefinerte typer (kap. 8)